

Applications of Artificial Neural Network

HW2 - Experimental Report

羅政昕 R37141329

1. Introduction and Objective

I will experiment with at least 5 different combinations as the HW instruction says, which will be hyperparameter adjustments based on a baseline model provided by the professor.

For the convenience of running the experiment, I had modified the baseline code a bit, making it a flexible model that can take inputs in a config fashion. For the baseline experiment, I still use the exact same hyperparameters as the provided baseline model.

In the next section, we will dive deeper and explore further the experiments I had conducted.

2. Experimental Setup

In this experiment, eight different hyperparameter settings were tested. The experiments changed hidden layer architecture, activation function, optimizer, and learning rate.

The tested optimizers include:

- SGD
- Momentum
- Adam

The tested activation functions include:

- Sigmoid
- ReLU

The result table is shown below:

Experiment	Hidden layers	Activation	Optimizer	LR	Epochs	Final train loss	Final val loss	Test accuracy
E6 Adam high LR 3 hidden	[32, 16, 8]	ReLU	Adam	0.020	100	0.0582	0.0654	0.9900
E6 Adam high LR	[16, 8]	ReLU	Adam	0.020	100	0.0636	0.0645	0.9833
E5 Adam	[16, 8]	ReLU	Adam	0.010	100	0.2447	0.2067	0.9733
E3 Deeper ReLU	[8, 8]	ReLU	SGD	0.010	100	0.6253	0.6178	0.8833
E4 Momentum	[8, 8]	ReLU	Momentum	0.010	100	0.3352	0.2994	0.8633
E6 Adam low LR	[16, 8]	ReLU	Adam	0.001	100	0.4560	0.4396	0.8567
E2 More neurons	[8]	Sigmoid	SGD	0.010	100	0.6639	0.6621	0.7067
E1 Baseline	[2]	Sigmoid	SGD	0.001	100	0.6871	0.6869	0.5000

3. Experiment Analysis

3-1. E1 Baseline: [2], Sigmoid, SGD, LR = 0.001

The baseline model achieved only 50.00% test accuracy (in most trials, sometimes lower sometimes higher), which is close to random guessing in a binary classification task.

The final train loss was 0.6871, and the final validation loss was 0.6869. Both losses remained high and very close to each other. This indicates that the model was not overfitting. Instead, it was strongly underfitting.

There are two main reasons for this poor result. First, the hidden layer only has two neurons, so the model capacity is too low to learn the non-linear moon-shaped boundary. Second, the learning rate is only 0.001, which makes SGD update the

model parameters very slowly. As a result, the model cannot effectively move toward a good solution within 100 epochs.

This experiment shows that the baseline architecture is too simple for the Moons Dataset.

3-2. E2 More Neurons: [8], Sigmoid, SGD, LR = 0.01

This experiment increased the hidden layer size from 2 neurons to 8 neurons and also increased the learning rate from 0.001 to 0.01.

The test accuracy improved from 50.00% to 70.67%. The final train loss decreased to 0.6639, and the final validation loss decreased to 0.6621.

This shows that increasing the number of neurons gives the model better representation ability. The model is now able to capture more information from the data than the baseline model.

However, the improvement is still limited. The main reason is that the hidden activation function is still Sigmoid. Sigmoid can suffer from gradient saturation, especially when the input values move into the flat regions of the function. When this happens, gradient values become small, and learning becomes slow. In addition, the optimizer is still plain SGD, which does not use adaptive learning rates or momentum.

Therefore, this model performs better than the baseline due to more neurons added, but still underfits the non-linear structure of the dataset.

3-3. E3 Deeper ReLU: [8, 8], ReLU, SGD, LR = 0.01

This experiment changed the architecture to two hidden layers, each with 8 neurons. The activation function was also changed from Sigmoid to ReLU.

The test accuracy improved to 88.33%, which is much better than the Sigmoid-based models. The final train loss was 0.6253, and the final validation loss was 0.6178.

The accuracy improvement shows that ReLU is more suitable than Sigmoid for this task. ReLU reduces the saturation problem and allows gradients to flow more effectively during backpropagation. Adding another hidden layer also increases the model's ability to represent complex non-linear decision boundaries.

However, the final losses are still relatively high. This means that although many predictions are correct after thresholding, the predicted probabilities may not be very confident. The optimizer is still plain SGD, so convergence is not as fast or stable as Adam.

This experiment shows that changing from Sigmoid to ReLU and increasing network depth can greatly improve classification accuracy, even without changing the optimizer.

3-4. E4 Momentum: [8, 8], ReLU, Momentum, LR = 0.01

This experiment used the same architecture as E3, but changed the optimizer from plain SGD to Momentum.

The test accuracy was 86.33%, which is slightly lower than E3's 88.33%. However, the final train loss decreased significantly to 0.3352, and the final validation loss decreased to 0.2994.

This result is interesting because the model has lower loss but slightly lower accuracy than E3. This can happen because binary cross-entropy loss and classification accuracy measure different things. Accuracy only checks whether the prediction is on the correct side of the 0.5 threshold. Loss also considers prediction confidence. Therefore, a model can have lower loss but slightly lower threshold-based accuracy.

Momentum generally helps optimization by accumulating past gradient direction. This allows the optimizer to move faster through shallow areas and reduce inefficient oscillations. In this case, Momentum clearly improves loss convergence compared with plain SGD, even though the final accuracy is slightly lower.

This experiment shows that Momentum improves the optimization process, but the final accuracy may still depend on the exact decision threshold and distribution of predicted probabilities.

3-5. E6 Adam Low LR: [16, 8], ReLU, Adam, LR = 0.001

This experiment used a stronger architecture with two hidden layers, [16, 8], and the Adam optimizer. However, the learning rate was set to a low value of 0.001.

The test accuracy was 85.67%. The final train loss was 0.4560, and the final validation loss was 0.4396.

Although Adam is generally more efficient than SGD, the learning rate in this experiment was too conservative. Because the learning rate was small, the model did not fully converge within 100 epochs. The result is better than the baseline and E2, but worse than the Adam experiments with higher learning rates. I also tried to do this experiment with more epochs (500, this is not listed), but the accuracy is still not as high as the Adam experiments that have $LR > 0.01$.

This experiment shows that even with a strong optimizer such as Adam, the learning rate still plays a major role. If the learning rate is too low, convergence becomes slow and the model may not reach its full performance within the epoch limit.

3-6. E5 Adam: [16, 8], ReLU, Adam, $LR = 0.01$

This experiment used the same [16, 8] architecture as the low learning rate Adam experiment, but increased the learning rate from 0.001 to 0.01.

The test accuracy increased significantly to 97.33%, exceeding the assignment target of 95%. The final train loss was 0.2447, and the final validation loss was 0.2067.

The large improvement shows that Adam with a suitable learning rate can converge much more effectively. The [16, 8] architecture provides enough capacity to represent the non-linear moon-shaped decision boundary, while ReLU helps the model avoid the saturation problem of Sigmoid.

The validation loss is lower than the training loss in this result. This may happen because the test/validation split is slightly easier than the training split, or because of random sampling variation. Since both train and validation results are good and the accuracy is high, there is no strong sign of overfitting.

This experiment is one of the successful configurations because it achieves high accuracy and stable learning.

3-7. E6 Adam High LR: [16, 8], ReLU, Adam, $LR = 0.02$

This experiment used the same architecture as E5 but increased the learning rate from 0.01 to 0.02.

The test accuracy improved further to 98.33%. The final train loss was 0.0636, and the final validation loss was 0.0645.

This is a very strong result. The training and validation losses are both low and very close to each other. This means the model learned the training data well while still generalizing effectively to the test set.

Compared with E5, the higher learning rate allowed Adam to converge faster and reach a better solution within 100 epochs. Unlike very large learning rates that may cause unstable training, $LR = 0.02$ still remained stable in this experiment.

This experiment shows that for the [16, 8] ReLU architecture, Adam with $LR = 0.02$ is more effective than Adam with $LR = 0.01$ or 0.001 .

3-8. E6 Adam High LR 3 Hidden: [32, 16, 8], ReLU, Adam, $LR = 0.02$

This is the best-performing experiment. It used three hidden layers with 32, 16, and 8 neurons, ReLU activation, Adam optimizer, learning rate 0.02, and 100 epochs.

The model achieved 99.00% test accuracy, which is the highest result among all experiments. The final train loss was 0.0582, and the final validation loss was 0.0654.

Compared with the two-hidden-layer Adam high LR model, this model slightly improved the accuracy from 98.33% to 99.00%. The additional hidden layer and larger first hidden layer gave the model more representational power. This allowed it to fit the curved decision boundary more precisely.

At the same time, the validation loss is only slightly higher than the training loss. The gap between training and validation loss is small, so there is no serious evidence of overfitting. This suggests that the architecture is more powerful but still generalizes well.

This experiment gives the best balance between fitting ability and generalization performance.

4. Comparison of Main Hyperparameter Effects

4.1 Effect of Hidden Layer Architecture

The results show that architecture has a strong effect on performance.

The baseline model with [2] hidden neurons achieved only 50.00% accuracy. Increasing the hidden layer to [8] improved accuracy to 70.67%. Using two ReLU hidden layers [8, 8] improved accuracy further to 88.33% with SGD.

The best model used three hidden layers [32, 16, 8] and achieved 99.00% accuracy. This shows that the Moons Dataset requires enough hidden capacity to learn a curved non-linear decision boundary.

However, more layers are not always automatically better. A larger model can overfit if the dataset is small or if training continues too long. In this experiment, the three-hidden-layer model still generalized well because the validation loss remained low and close to the training loss.

4.2 Effect of Activation Function

The Sigmoid models performed worse than the ReLU models.

The Sigmoid baseline achieved 50.00%, and the larger Sigmoid model achieved 70.67%. After switching to ReLU, the accuracy increased significantly.

ReLU is more suitable in this experiment because it reduces the vanishing gradient problem. Sigmoid can saturate when inputs are too large or too small, causing gradients to become very small. This makes learning slower. ReLU keeps stronger gradients for positive inputs and allows deeper networks to train more effectively.

Therefore, ReLU is a better hidden activation function for this BPNN classification task.

4.3 Effect of Optimizer

The optimizer also strongly affected the result.

Plain SGD worked poorly in the baseline and Sigmoid experiments. It improved when combined with ReLU and deeper architecture, but still did not reach the 95% target.

Momentum reduced the loss significantly compared with SGD, showing that momentum-based updates can improve convergence. However, its final accuracy was still lower than the Adam-based models.

Adam produced the best results. With [16, 8], ReLU, and LR = 0.01, Adam achieved 97.33% accuracy. With LR = 0.02, it achieved 98.33%. With three hidden layers [32, 16, 8], it achieved 99.00%.

This shows that Adam is the most effective optimizer in this experiment because it adapts learning rates for individual parameters and converges faster within the same 100 epochs.

4.4 Effect of Learning Rate

The learning rate comparison is especially clear among the Adam experiments.

Architecture	Optimizer	LR	Accuracy
[16, 8]	Adam	0.001	85.67%
[16, 8]	Adam	0.010	97.33%
[16, 8]	Adam	0.020	98.33%

The low learning rate of 0.001 was too slow for 100 epochs. Increasing it to 0.01 greatly improved convergence. Increasing it again to 0.02 further improved the result.

Therefore, LR = 0.02 was the best learning rate among the tested Adam configurations.

However, this does not mean that a higher learning rate is always better. If the learning rate becomes too large, the model may become unstable or fail to converge. In this experiment, 0.02 was high enough to speed up training but still stable enough for Adam.

5. Optimal Parameter Combination

The optimal parameter combination found in this experiment is:

Parameter	Final choice	Justification
Hidden layers	[32, 16, 8]	More neurons, deeper layers, better results.
Activation function	ReLU	ReLU performs the best on the moons dataset.
Optimizer	Adam	Adam performs the best on the moons dataset.
Learning rate	0.02	A slightly bigger learning rate yields better accuracy.
Epochs	100	Good balance between time and results.
Final train loss	0.0582	
Final validation loss	0.0654	
Test accuracy	99.00%	

6. Bias-Variance Tradeoff Analysis

The best model, [32, 16, 8] + ReLU + Adam + LR = 0.02, shows a good balance between bias and variance.

The baseline model has high bias. It is too simple and cannot fit the non-linear Moons Dataset. This is shown by its low accuracy and high training loss.

The best model has much lower bias because it has enough layers and neurons to learn the curved decision boundary. The three hidden layers give the model stronger non-linear representation ability.

At the same time, the model does not show serious overfitting. The final train loss is 0.0582, and the final validation loss is 0.0654. The validation loss is only slightly higher than the training loss. This small gap suggests that the model generalizes well. Therefore, the model successfully reduces underfitting without creating severe overfitting.

7. Training and Validation Loss Curve Analysis

For the best model (Figure 1), the training and validation losses show the following traits:

- Training loss decreases steadily.
- Validation loss also decreases steadily.
- The two curves remain close to each other.
- There is no large gap between training and validation loss.
- The final validation loss remains very low.

These indicate that the model converges smoothly and generalizes well.

Compared with the baseline model (Figure 2), the difference is clear. The baseline loss remains high and does not improve much, showing underfitting and slow convergence. The best Adam model quickly reduces both training and validation loss, showing much stronger optimization.

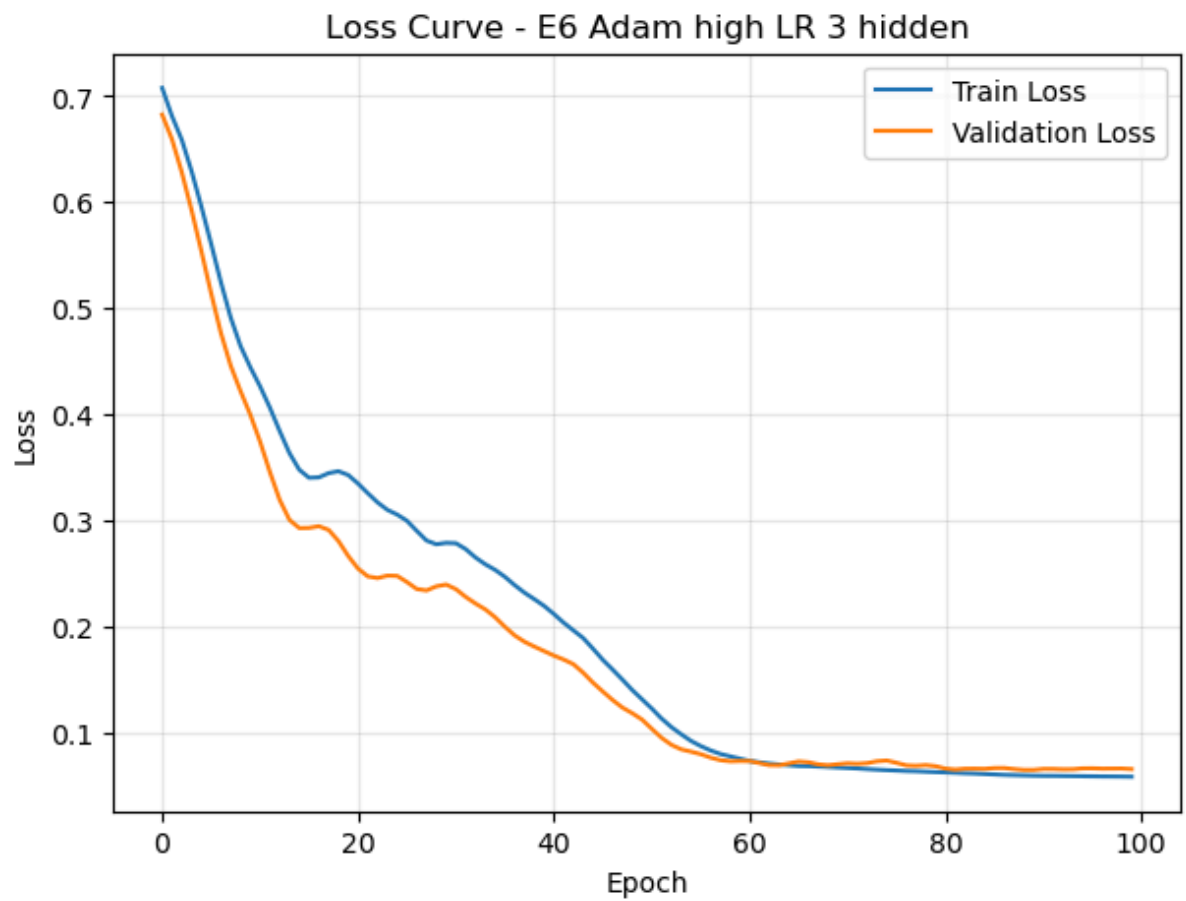


Figure 1. Training and validation loss curve of the optimal model

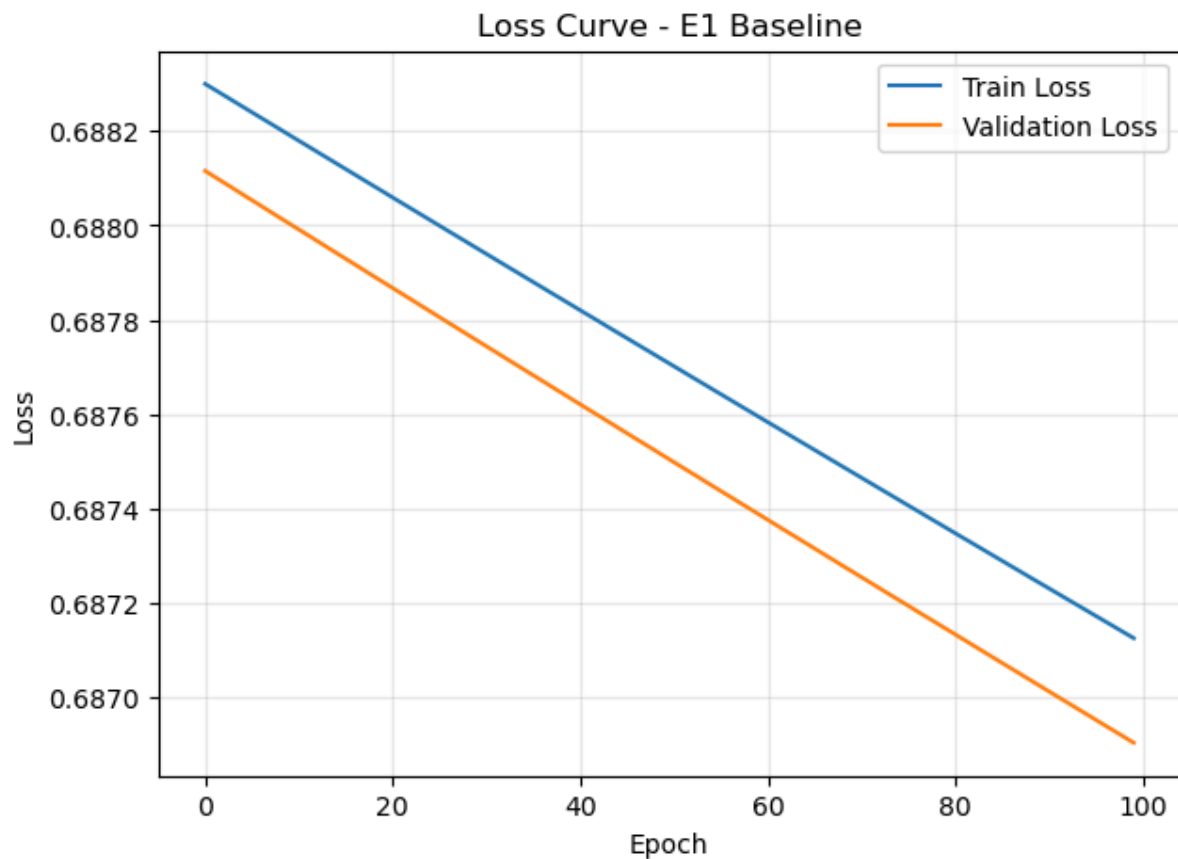


Figure 2. Training and validation loss curve of the baseline model

8. Decision Boundary Analysis

The Moons Dataset requires a non-linear decision boundary. The best model produces a decision boundary that follows the shape of the two classes closely.

This confirms that the three-hidden-layer ReLU network successfully learned the non-linear structure of the dataset.

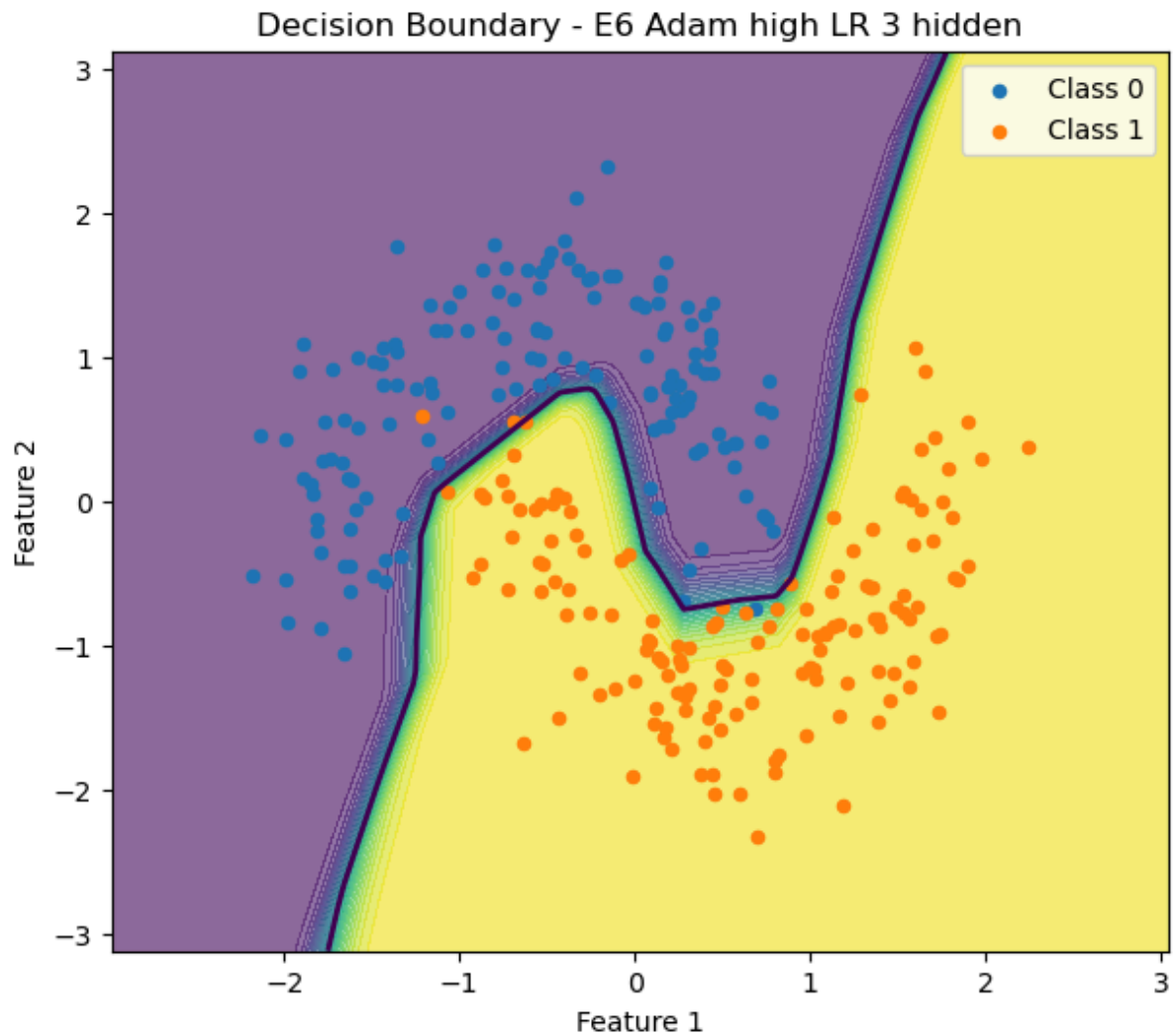


Figure 3. Decision boundary of the optimal model

9. Critical Thinking: Would the Same Optimal Parameters Work on Other Datasets?

The same optimal parameter combination may not produce the same optimization effect on other datasets.

The best configuration in this experiment was found specifically for the Moons Dataset. The Moons Dataset has a two-dimensional, non-linear, curved class distribution. Therefore, a multi-layer ReLU network is suitable.

However, different datasets may require different model structures.

Example 1: Circle Dataset

A circle dataset is also non-linear, so the model may still perform well. However, the decision boundary shape is different from the Moons Dataset. The best number of layers, neurons, and learning rate may change. A model that works well for moon-shaped data may not be exactly optimal for circular data.

Example 2: Linearly Separable Dataset

For a linearly separable dataset, this model may be unnecessarily complex. A simple linear classifier or a neural network without hidden layers may already achieve high accuracy. Using [32, 16, 8] hidden layers may waste computation and may increase the risk of overfitting.

Example 3: Regression Problem

For a regression problem, this exact model would not be appropriate. The current model uses a Sigmoid output and binary cross-entropy loss, which are designed for binary classification. For regression, the output layer should usually be linear, and the loss function should be changed to something like mean squared error.

Therefore, the optimal parameter combination is task-dependent. It should be adjusted according to the dataset distribution, problem type, and model behavior.

10. Conclusion

This experiment shows that BPNN performance is strongly affected by hidden layer architecture, activation function, optimizer, and learning rate.

The baseline model performed poorly because it had too little capacity and used a very small learning rate. Increasing the number of neurons improved performance, but Sigmoid activation and plain SGD were still not sufficient. ReLU activation significantly improved learning because it reduced the gradient saturation problem. Momentum improved loss convergence compared with plain SGD, while Adam produced the best overall performance.

Among all experiments, the best model was:

[32, 16, 8] hidden layers + ReLU activation + Adam optimizer + learning rate 0.02

This model achieved:

- Final train loss: 0.0582
- Final validation loss: 0.0654

- Test accuracy: 99.00%

The model successfully achieved near-perfect separation on the Moons Dataset and exceeded the assignment target of 95% accuracy. The small gap between training and validation loss suggests good generalization and no severe overfitting.

Overall, the results confirm that a properly designed BPNN can effectively solve non-linear classification problems when suitable architecture, activation function, optimizer, and learning rate are selected.